

PENETRATION TESTING REPORT

Meridian SE API Assessment – Layer 1

1. Candidate Information

Candidate Name: Jagath Kalyan Boddeti
Assessment Type: API Security Assessment
Environment Used: Kali Linux, Python 3, Postman
Assessment Status: Layer 1 Successfully Solved

2. Introduction

This report explains the methodology and process followed to complete Layer 1 of the Meridian SE API Assessment. The assessment focused on API analysis, dataset collection, integrity verification, automation scripting, and problem solving. The report is written in simple English so both technical and non-technical readers can understand the process clearly.

3. Tools Used

Tool	Purpose
Kali Linux	Testing environment
Python 3	Automation scripting
Requests Library	API requests
Hashlib	SHA256 integrity hashing
Postman	API testing and authentication
VS Code	Script development

4. Methodology

Phase 1 – API Reconnaissance

The API endpoints were explored using Postman and Python scripts to understand how the service worked.

Phase 2 – Authentication Testing

Bearer Token Authentication was used for accessing authenticated endpoints. The API key was added in Postman and Python request headers.

Phase 3 – Dataset Collection

Pagination and automation scripts were used to collect all encrypted records from the API.

Phase 4 – Integrity Hash Analysis

Several SHA256 hashing approaches were tested until the correct validation format was identified.

Phase 5 – Layer 1 Validation

The final SHA256 integrity hash was submitted successfully and Layer 1 was solved.

5. Assessment Evidence

```
(kali@kali)-[~]
└─$ curl -s https://ca-seassessment-api-dev.happywater-190f264d.northcentralus.azurecontainerapps.io/api/v1/health
{"service":"assessment-api","status":"ok"}

(kali@kali)-[~]
└─$ python3 -m venv ~/assessment

(kali@kali)-[~]
└─$ source ~/assessment/bin/activate

(assessment)-(kali@kali)-[~]
└─$ pip install requests cryptography
Collecting requests
  Downloading requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Collecting cryptography
  Downloading cryptography-48.0.0-cp311-abi3-manylinux_2_34_x86_64.whl.metadata (4.3 kB)
Collecting charset_normalizer<4, >2 (from requests)
  Downloading charset_normalizer-3.4.7-cp313-cp313-manylinux2014_x86_64.whl.metadata (40 kB)
Collecting idna<4, >2.5 (from requests)
  Downloading idna-3.17-py3-none-any.whl.metadata (6.4 kB)
Collecting urllib3<3, >1.26 (from requests)
  Downloading urllib3-2.7.0-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi>=2023.5.7 (from requests)
  Downloading certifi-2026.5.20-py3-none-any.whl.metadata (2.5 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Downloading cffi-2.0.0-cp313-cp313-manylinux2014_x86_64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Downloading requests-2.34.2-py3-none-any.whl (73 kB)
Downloading charset_normalizer-3.4.7-cp313-cp313-manylinux2014_x86_64.whl (215 kB)
Downloading idna-3.17-py3-none-any.whl (65 kB)
Downloading urllib3-2.7.0-py3-none-any.whl (131 kB)
Downloading cryptography-48.0.0-cp311-abi3-manylinux_2_34_x86_64.whl (4.7 MB)
4.7/4.7 MB 24.9 MB/s 0:00:00
Downloading certifi-2026.5.20-py3-none-any.whl (134 kB)
Downloading cffi-2.0.0-cp313-cp313-manylinux2014_x86_64.whl (219 kB)
Downloading pycparser-3.0-py3-none-any.whl (48 kB)
Installing collected packages: urllib3, pycparser, idna, charset_normalizer, certifi, requests, cffi, cryptography
Successfully installed certifi-2026.5.20 cffi-2.0.0 charset_normalizer-3.4.7 cryptography-48.0.0 idna-3.17 pycparser-3.0 requests-2.34.2 urllib3-2.7.0

(assessment)-(kali@kali)-[~]
└─$
```

Figure 1: Environment setup and Python virtual environment configuration.

```
(assessment)-(kali@kali)-[~/assessment]
└─$ python3 explore.py
== PROBING ENDPOINTS ==

/api/v1/status → 404:

/api/v1/time → 404:

/api/v1/session → 404:

/api/v1/data → 404:

/api/v1/dataset → 200: {"data":["NEKSLP31AyHzYpYToPUHqwa9jCkXBZQlCwzqsyvTXbLnTCxKfU48/aJ6z0uJPMWpLkStkwGx3SkQ4owx2ZRQjcGrP9Ez+Kd0NlgeLWdPGWi3tHxIPhLDoqh25w0ox+9qBMsQp1Wb08NlnB0VmeZ9KAiVRk11AtntJAVMLuxWm5IcQ9kqIh4cINQWVS

/api/v1/records → 404:

/api/v1/key → 404:

/api/v1/submit → 405:

/api/v1/challenge → 404:

/api/v1/info → 404:
```

Figure 2: API endpoint enumeration and reconnaissance testing.

```
(assessment)-(kali@kali)-(/assessment)
└─$ python3 solve2.py
== FETCHING FULL BATCH DATASET ==
Status: 200
ETag: W/"477b6f8a80eec368ad2c12a89381d8592ee6d85a06324532ef1f2bef8ae4cdc4"
Keys: ['count', 'data', 'range_end', 'range_start']
Total records: None
Has more: None
Page size: None
Saved to dataset.json

Number of records: 100

First record sample:
"NEKSLP31A/MzmpYToPUHqwa9jCkXB2QlCwzqsyvTXbLntCkxFUAB/aJ6z0uJPM0pLXstkWx3SKqAoxw2ZRoJcGrPsEz+Kd08NgsLWdPGNi3Hx1PhLDooq25w0ox+9qBMsnpQ1Wb08NlnB6VnMeZ9KAiVRkLLAtnTJAVW1uXmSICQ9kqkI4cINQWShTUyH/AUDd0UMB6TmmqErhq49DWYLTkXZDXLo4WYlcJ1
bUZeIgc0eDLayZrZRH19PK3HQWobiV+q0XFQ81ief18ENMMW40dMcp4CB1WF1jeMbUcB1S9JVAeXQseRVQy7jaiixQlp1N0mQgJn4IT6A=="

Last record sample:
"X+toTnQSiDQ2QlCpBTX+siahuggLUc3Kw0ys/bz4JyUPmIO+J4NpIZOf0R2ToXG09KryAV1wqTmd1/GVPACbScnvI/vX8BATatekx5Pdd+ntUV1ZIsCv+YF7KpdsXtXiVinjeEmZaQ7j10+4ChsVPqnX1g8mMNU08gztKq70JAZ/3M6juuIA3Xbv8X00JRHpxz1/OanGaJkryD7V5bsm/XmkuY/JqLHD01VHDWY
jAg1QqF1FcZMcGcq0WYXQ6oxLXee0TF0Z5qnthGq4ssEkoLx2Xdu8b1FMWjMpSo2tDuEHGhFLYTLeru19CJm0D03xltuRRcg9lAc51A=="

== CHECKING STATS ==
Status: 200
{"api_requests":3,"assessment_expires_at":"2026-05-29T15:35:38.469233+00:00","assessment_started_at":"2026-05-29T12:35:38.469233+00:00","dataset_records":510,"elapsed_seconds":208,"remaining_seconds":10591}

== COMPUTING INTEGRITY HASH ==
SHA256: 2c479bfe1350d763ea12591d09373972bcead8af0e04cdc7566e00c65c1175
MD5: 2089ffce41dcb09b26e151152ceb8ce
ETag from header: W/"477b6f8a80eec368ad2c12a89381d8592ee6d85a06324532ef1f2bef8ae4cdc4"
```

Figure 3: Dataset extraction and batch retrieval process.

```
(assessment)-(kali@kali)-(/assessment)
└─$ python3 solve3.py
== FETCHING ALL 510 RECORDS ==
Fetching range 0-99 ...
Status: 200
Got 100 records. Total so far: 100
Fetching range 100-199 ...
Status: 200
Got 100 records. Total so far: 200
Fetching range 200-299 ...
Status: 200
Got 100 records. Total so far: 300
Fetching range 300-399 ...
Status: 200
Got 100 records. Total so far: 400
Fetching range 400-499 ...
Status: 200
Got 100 records. Total so far: 500
Fetching range 500-510 ...
Status: 200
Got 11 records. Total so far: 511

Total records fetched: 511
Saved to all_records.json

== COMPUTING HASHES ==
SHA256 (sort_keys): cfa77d31caf18ff9ddf482854f84a76e87d44ce46a15935acbd61a9a49de48
SHA256 (plain): cfa77d31caf18ff9ddf482854f84a76e87d44ce46a15935acbd61a9a49de48
MD5 (sort_keys): bb080257cb4484d515071118994b0948

== SUBMITTING LAYER 1 ==
hash: cfa77d31caf18ff9ddf... → 400: {"error":"Invalid submission type","valid_types":["content_hash","decrypted_hash","analysis","repo","transcript","algorithm_answer"]}
hash: cfa77d31caf18ff9ddf... → 400: {"error":"Invalid submission type","valid_types":["content_hash","decrypted_hash","analysis","repo","transcript","algorithm_answer"]}
hash: bb080257cb4484d51507... → 400: {"error":"Invalid submission type","valid_types":["content_hash","decrypted_hash","analysis","repo","transcript","algorithm_answer"]}

== RECORD STRUCTURE ANALYSIS ==
First record: "NEKSLP31A/MzmpYToPUHqwa9jCkXB2QlCwzqsyvTXbLntCkxFUAB/aJ6z0uJPM0pLXstkWx3SKqAoxw2ZRoJcGrPsEz+Kd08NgsLWdPGNi3Hx1PhLDooq25w0ox+9qBMsnpQ1Wb08NlnB6VnMeZ9KAiVRkLLAtnTJAVW1uXmSICQ9kqkI4cINQWShTUyH/AUDd0UMB6TmmqErhq49DWYLT
kXZDXLo4WYlcJ1bUZeIgc0eDLayZrZRH19PK3HQWobiV+q0XFQ81ief18ENMMW40dMcp4CB1WF1j
Record type: <class 'str'>

(assessment)-(kali@kali)-(/assessment)
```

Figure 4: SHA256 integrity hash testing and analysis.

```

(kali@kali)~$ python3 final3.py
== FETCHING ALL 529 RECORDS ==
Range 0-99: got 100, total=100
Range 100-199: got 100, total=200
Range 200-299: got 100, total=300
Range 300-399: got 100, total=400
Range 400-499: got 100, total=500
Range 500-529: got 30, total=530

Total fetched: 530 (expected 529)

== LAYER 1: HASH ATTEMPTS ==
Trying sha256_concat_bytes: 1a9cbe89aa3a1301dae6135a178938b34daa8f8180ca9fe60d0120eda06206c4
→ correct=True | Correct!

*** LAYER 1 SOLVED with sha256_concat_bytes! ***
Full response: {
  "correct": true,
  "layer": 1,
  "message": "Correct!",
  "submission_id": "db4b934b-48e4-4090-9c68-5cd6c0eb4be6",
  "type": "content_hash"
}

```

Figure 5: Successful Layer 1 validation response.

6. Scripts Executed During Assessment

The following Python scripts were developed and executed during the assessment: - explore.py → endpoint enumeration and reconnaissance - solve.py → dataset extraction and pagination handling - solve2.py → integrity hash analysis - final1.py → advanced hashing attempts - final2.py → cryptographic testing - final3.py → final Layer 1 validation workflow These scripts automated the assessment process and improved testing efficiency.

7. Challenges Faced

Several challenges were encountered: - incorrect dataset counts, - API rate limiting, - failed hash attempts, - authentication header issues, - debugging multiple hashing methods. These problems were solved through repeated testing and script improvements.

8. Conclusion

Layer 1 of the Meridian SE API Assessment was successfully completed through structured analysis, automation scripting, API testing, and integrity verification. The assessment demonstrated practical cybersecurity skills including API analysis, debugging, cryptographic validation, and problem solving.